



ADDIS ABABA
**SCIENCE AND
TECHNOLOGY**
UNIVERSITY
UNIVERSITY FOR INDUSTRY

College of Engineering

Software Engineering

Database Systems: [SWEG2108]

**Title: Database Management System for Gebar
Commercial Bank**

No.	Name	Student ID No.
1.	Elabem Medfer	ETS 0471/17
2.	Elsabet Berhanu	ETS 048/17
3.	Enjifeto Abdella	ETS 0497/17
4.	Ermias Berhane	ETS 0504/17
5.	Ewenetu Haile	ETS 0528/17
6.	Kasim Muhdin	ETS 0814/17

Summited to: Instructor Yaynshet Medihn

Submission Date: 22/05/2026

Table of Contents

Chapter 1: Introduction	2
1.1 Background of the Organization.....	2
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope of the Project	3
1.5 Benefits of the Proposed System	3
1.6 Methodology and Tools	3
Chapter 2: Database Design	3
2.1 Identification of Entities and Relationships	3
2.2 ERD Diagram	4
2.3 Logical Schema	5
2.4 Normalization	5
2.5 Design of Inputs (Forms), Outputs, and Reports	9
Chapter 3: Implementation	10
3.1 Microsoft Server SQL Implementation (Relational Database)	11
3.2 MongoDB Implementation (NoSQL Database)	11
3.3 Sample Data	12
3.4 Testing	12
3.5 Results, Challenges & Future Improvements	12
References	13
Annex.....	14
Annex A: Survey Questions	14
Annex B: Sample Forms	16
Annex C: Queries	18

Chapter 1: Introduction

1.1 Background of the Organization

Gebbar Commercial Bank is a simulated mid-sized commercial bank operating in Ethiopia with its headquarters in Addis Ababa. The bank provides a wide range of financial services to individual and corporate customers, including savings accounts, current accounts, fixed deposits, personal and business loans, and inter-branch fund transfers.

The bank currently manages several branches and serves thousands of customers. As customer numbers and transaction volumes grow, the need for a robust, automated database system has become critical.

1.2 Problem Statement

The existing semi-manual data management system at Gebbar Commercial Bank suffers from data redundancy, lack of integrity, slow information retrieval, difficulties in tracking multi-branch transactions, and delays in loan processing and reporting. These issues result in operational inefficiencies, increased error risk, and inadequate support for managerial decision-making. We used survey questions to gather information about these issues. For the whole survey questions refer the Annex A section (/annex/survey_questions.pdf).

1.3 Objectives

General Objective

To design and implement a comprehensive Bank Management System using both relational (Microsoft SQL Server) and NoSQL (MongoDB) technologies to improve the efficiency, accuracy, and scalability of core banking operations.

Specific Objectives

- To analyze data requirements and identify entities and relationships.
- To develop a conceptual ER model and normalize it to BCNF.
- To implement the system in Microsoft SQL Server and MongoDB.
- To design input forms and output reports.
- To test the system and document results and improvements.

1.4 Scope of the Project

The system covers customer management, savings/current accounts, transaction processing, loan management, and basic branch/employee operations. Advanced features such as online banking, ATM integration, and full security modules are excluded.

1.5 Benefits of the Proposed System

- Elimination of data redundancy and anomalies.
- Improved transaction speed and report accuracy.
- Enhanced data consistency across branches.
- Better decision support through timely reports.
- Scalability using a hybrid database approach.

1.6 Methodology and Tools

The project followed a structured database development lifecycle: requirements analysis, conceptual design, logical design, normalization, implementation, and testing.

Tools Used: Microsoft SQL Server, MongoDB, Draw.io (ER diagrams), Microsoft Office, Microsoft Powerpoint, Telegram, and GitHub.

Chapter 2: Database Design

2.1 Identification of Entities and Relationships

The following entities and attributes were identified:

1. **Branch:** branch_id (PK), branch_name, address, phone_number, manager_id (FK).
2. **Customer:** customer_id (PK), first_name, last_name, phone_number, registration_date, email, national_id, branch_id (FK).
3. **Account:** account_no (PK), customer_id (FK), account_type, balance, open_date, status, branch_id (FK).

4. **Loan:** loan_id (PK), customer_id (FK), branch_id (FK), loan_type, loan_amount, interest_rate, duration_months, approval_date, status, application_date, disbursement_date, monthly_installment, outstanding_balance, next_due_date.
5. **Transaction:** transaction_id (PK), account_no (FK), transaction_type, amount, transaction_date, description, reference_number, branch_id (FK).
6. **Employee:** employee_id (PK), first_name, last_name, position, salary, email, phone_number, hire_date, branch_id (FK).
7. **Loan Payments:** payment_id (PK), payment_date, amount_paid, principal_paid, interest_paid, remaining_balance, employee_id (FK), loan_id (FK).
8. **Users:** user_id (PK), user_name, password, role, status, employee_id (FK), customer_id (FK), last_login.
9. **Audit Log:** log_id (PK), table_name, record_id, old_values, new_values, action, changed_at, description, employee_id (FK).

Key Relationships (Cardinality):

- Branch -> Customer (1:M)
- Branch -> Employee (1:M)
- Branch -> Transaction (1:1)
- Branch -> Account (1:1)
- Branch -> Loan (1:M)
- Customer -> Loan (1:M)
- Customer -> Account (1:M)
- Loan -> Loan Payments (1:M)

2.2 Entity-Relationship (ER) Diagram

The Entity-Relationship Diagram (ERD) represents the conceptual design of the Gebar Commercial Bank Management System. It identifies the main entities and their relationships.

Key Entities:

- Branch
- Customer
- Employee

- Account
- Loan
- Transaction
- Loan_Payment
- User

Major Relationships:

- A **Branch** manages many **Customers**, **Employees**, **Accounts**, and **Loans** (One-to-Many).
- A **Customer** can have multiple **Accounts** and **Loans** (One-to-Many).
- A **Loan** can have multiple **Loan_Payments** (One-to-Many).
- An **Account** can be involved in multiple **Transactions** (One-to-Many).
- **Employees** can record **Transactions** and **Loan_Payments**.
- **Users** are linked to either **Employees** (for staff) or **Customers** (for online access).

The ER Diagram clearly shows these relationships using proper cardinality (1:1, 1:N where applicable). The complete ER Diagram is provided in /diagrams/erd.png.

2.3 Logical Schema (Relational Model)

The conceptual ER model was mapped into a logical relational schema. The following tables represent the final structure after applying normalization rules up to BCNF:

Main Tables:

- **Branch** (branch_id PK)
- **Customer** (customer_id PK)
- **Employee** (employee_id PK)
- **Account** (account_no PK)
- **Loan** (loan_id PK)
- **Transaction** (transaction_id PK)
- **Loan_Payment** (payment_id PK)
- **User** (user_id PK)
- **Audit_Log** (log_id PK)

All relationships were implemented using appropriate **primary keys** and **foreign keys**. The schema ensures referential integrity and eliminates redundancy. Detailed table structures with column names, data types, keys, and descriptions are provided in the Normalization Report (/diagrams/normalization.pdf).

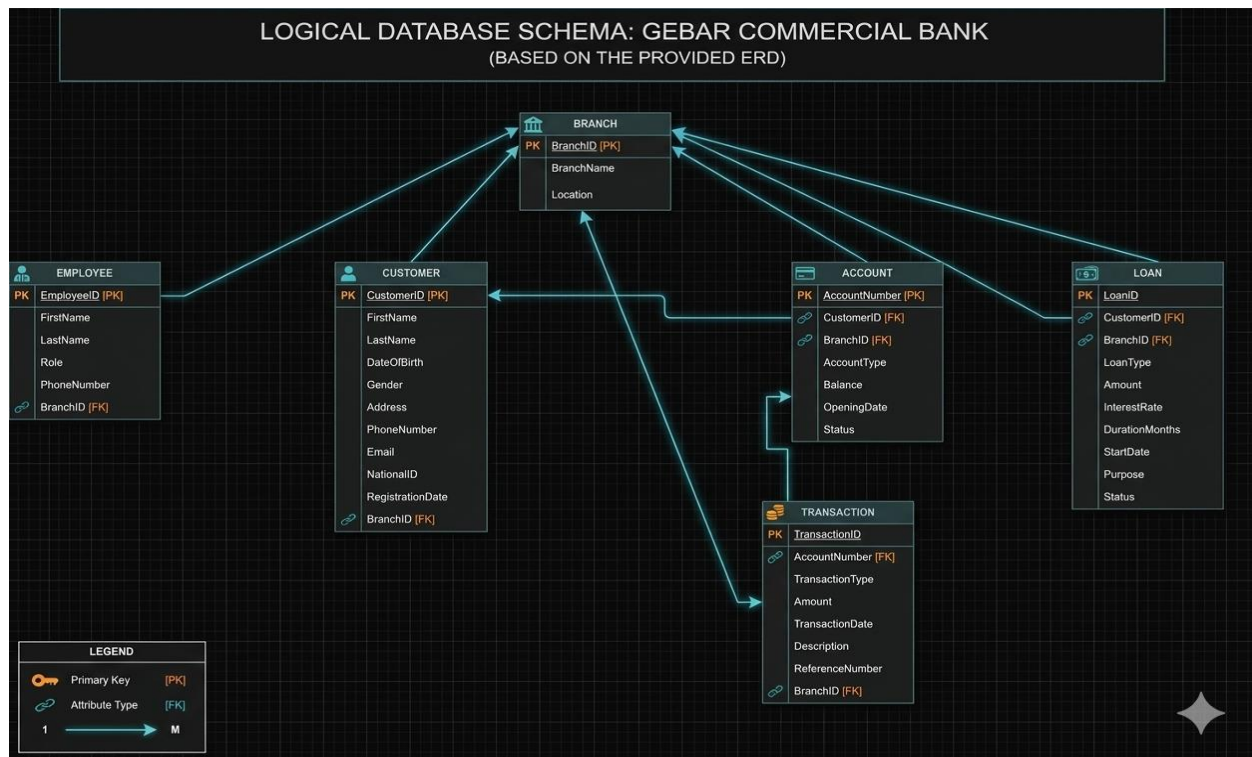


Figure 2.1 Logical Schema

2.4 Normalization

Normalization is a systematic process of organizing data in the relational database to minimize redundancy, eliminate data anomalies (insertion, update, and deletion anomalies), and ensure data integrity. In this Bank Management System for Gebar Commercial Bank, the database was normalized from Unnormalized Form (UNF) up to **Boyce-Codd Normal Form (BCNF)**.

The detailed step-by-step normalization process, including functional dependency analysis and decomposition, is provided in the separate document </diagrams/normalization.pdf>.

2.4.1 Normalization Process Summary

The normalization was carried out in the following stages:

- **Unnormalized Form (UNF):** All banking data was initially stored in a single flat table with repeating groups (e.g., multiple accounts, loans, and transactions per customer) and multi-valued attributes.
- **First Normal Form (1NF):** Eliminated repeating groups and ensured atomic values by creating separate relations for Customer, Account, Loan, Transaction, Branch, Employee, Loan Payments, Users and Audit Log.

- **Second Normal Form (2NF):** Since all tables used single-column primary keys, there were no partial dependencies. All non-key attributes fully depend on their respective primary keys.
- **Third Normal Form (3NF):** Removed any transitive dependencies. Attributes such as branch details were moved to a separate Branch table, and employee details were isolated from other entities.
- **Boyce-Codd Normal Form (BCNF):** Verified that every determinant in the functional dependencies is a candidate key. No BCNF violations were found.

2.4.2 Final Normalized Schema (BCNF)

The final relational schema after achieving BCNF consists of the following nine tables:

1)Branch

- branch_id (PK)
- branch_name
- address
- phone_number
- manager_id (FK)

2)Customer

- customer_id (PK)
- first_name
- last_name
- phone_number
- registration_date
- email
- national_id
- branch_id (FK)

3)Account

- account_no (PK)
- customer_id (FK)
- account_type
- balance
- open_date
- status
- branch_id (FK)

4)Loan

- loan_id (PK)
- customer_id (FK)
- branch_id (FK)
- loan_type
- loan_amount
- interest_rate
- duration_months
- approval_date
- status
- application_date
- disbursement_date
- monthly_installment
- outstanding_balance
- next_due_date

5)Transaction

- transaction_id (PK)
- account_no (FK)
- transaction_type
- amount
- transaction_date
- description
- reference_number
- branch_id (FK)
- loan_id (FK)

6)Employee

- employee_id (PK)
- first_name
- last_name
- position
- salary
- email
- phone_number
- hire_date
- branch_id (FK)

7) Loan Payments

- payment_id (PK)
- payment_date
- amount_paid
- principal_paid
- interest_paid
- remaining_balance
- employee_id (FK)
- loan_id (FK)

8) Users

- user_id (PK)
- user_name
- password
- role
- status
- employee_id (FK)
- customer_id (FK)
- last_login

9) Audit Log

- log_id (PK)
- table_name
- record_id
- old_values
- new_values
- action
- changed_at
- description
- employee_id (FK)

2.4.3 Functional Dependencies (Summary)

All functional dependencies were analyzed and preserved. The primary keys are the sole determinants in non-trivial dependencies, satisfying BCNF requirements.

2.4.4 Benefits of Normalization

- Elimination of redundancy (e.g., branch and customer details are not repeated).
- Prevention of update, insertion, and deletion anomalies.
- Improved data consistency and integrity.
- Easier maintenance and better query performance.
- Clear separation of concerns between entities.

The complete detailed normalization steps, including all intermediate forms, functional dependency tables, and before/after comparisons, are available in </diagrams/normalization.pdf>.

2.5 Design of Inputs (Forms), Outputs, and Reports

This section describes the input forms, outputs, and reports designed for the Bank Management System of Gebar Commercial Bank. The detailed layouts, screenshots, and sample filled versions are provided in the Annex section (/annex/sample_forms.pdf).

2.5.1 Input Forms

The system includes well-structured input forms to facilitate efficient data entry while maintaining data integrity. The major forms designed are:

- **Branch Registration Form** – For adding new branches to the banking system.
- **Customer Registration Form** – For adding new customers to the banking system.
- **Employee Registration Form** – For adding new bank employees.
- **Account Creation Form** – Used to create new customer accounts (Savings or Current).
- **Deposit Form** – For recording deposit financial transactions into customer accounts.
- **Withdrawal Form** – For recording withdrawal financial transactions from customer accounts.
- **Loan Application Form** – For processing loan requests from customers.
- **Transaction Table** – For logging or reviewing comprehensive transaction specs including reference numbers and descriptive tags.
- **Loan Payments Table** – For tracking and recording payment inputs regarding outstanding loan debts.
- **Users Table** – For recording and creating backend user authorization access profiles and credentials.
- **Audit_log Table** – For tracking and documenting structural modification inputs and system security actions

All forms incorporate appropriate validation rules, dropdown selections, and auto-generation of IDs where applicable.

2.5.2 Outputs and Reports

The system provides various outputs and management reports to support decision-making and daily operations. The key reports include:

- **Account Statement Report** – Displays detailed transaction history for a specific account over a selected period.
 - **Customer Information Report** – Provides a comprehensive list of all customers and their basic details.
 - **Loan Summary Report** – Shows the status and details of all loans issued by the bank.
 - **Employee List Report** – Displays information about all bank employees.
 - **Active Accounts Report** – Lists all currently active accounts with their balances.
-

Chapter 3: Implementation

This chapter describes the independent implementation of the Gebar Commercial Bank Management System using two different database technologies: **Microsoft SQL Server** (Relational) and **MongoDB** (NoSQL). Both systems were developed separately to demonstrate the strengths of each paradigm for the same banking domain.

3.1 Microsoft Server SQL Implementation (Relational Database)

Microsoft Server SQL was used to implement a fully normalized relational database.

- **Database Name:** gebar_bank_db
- **Number of Tables:** 9 (Branch, Customer, Account, Loan, Transaction, Employee, Loan Payments, Users, and Audit Log)
- All tables were created according to the BCNF schema designed in Chapter 2.
- Primary keys, foreign keys, and appropriate constraints were enforced.

Implementation Files:

- Schema creation script: /mysql/schema.sql
- Queries and operations: /mysql/queries.sql

3.2 MongoDB Implementation (NoSQL Database)

MongoDB was implemented separately as a document-oriented database for the same Bank Management System.

- **Database Name:** gebar_bank
- **Main Collections:** branch, customer, account, loan, transaction, employee, loan payments, users, and audit log

Key Characteristics:

- Used flexible document structure
- Embedded documents where appropriate (e.g., transaction history)
- Powerful aggregation pipelines for reporting

Implementation Files:

- Collection structures and sample documents: /mongodb/collections.json
- Queries and operations: /mongodb/queries.js

Sample Screenshots: *(Insert images here: MongoDB collections view and query outputs)*

3.3 Sample Data

Sample data was populated independently in both databases to test all major functionalities such as account operations, loan management, and transaction processing.

3.4 Testing

Both implementations were tested separately for:

- CRUD operations
- Data integrity
- Query performance
- Report generation

3.5 Results, Challenges & Future Improvements

Results:

- Both MySQL and MongoDB successfully support core banking operations.
- MySQL excels in structured data and complex relationships.
- MongoDB provides more flexibility for document storage and complex queries.

The screenshot shows a VS Code editor with a terminal window open. The terminal is running a MongoDB shell (mongosh) and displaying the output of several queries. The queries are numbered 5.5 through 5.9, followed by section 6.1. The output shows the results of various banking operations, including deposits, withdrawals, transfers, and loan applications. The terminal window is titled 'mongosh' and shows the current log ID as 'p6a0ff2fc4d9721eed4abc1135'.

```

mongodb > JS queries.js > ...
188 function parseDates(doc, dateFields) {
189   const out = Object.assign({}, doc);
PROBLEMS OUTPUT TERMINAL PORTS
> TERMINAL
Current Mongosh Log ID:p6a0ff2fc4d9721eed4abc1135>
- 5.5 DEPOSIT - update balance + insert transaction -
  Deposited 10000 ETB. New balance: 15000 ETB.

- 5.6 WITHDRAWAL - debit with balance guard -
  Withdrawn 5000 ETB. New balance: 10000 ETB.

- 5.7 TRANSFER between accounts -
  Transferred 15000 ETB from A1008 to A1002. Ref: TRF-2026-888

- 5.8 LOAN APPLICATION -
  Loan L006 for C012 submitted.

- 5.9 APPROVE loan -
  Loan L006 status changed to Approved.
SECTION 6 - AGGREGATION PIPELINES

- 6.1 Account Statement for A1001 (latest 10 with running balance) -
[
  {
    transaction_id: 'T006',
    transaction_type: 'Withdrawal',
    amount: 5000,
    transaction_date: ISODate('2026-05-22T06:18:36.131Z'),
    description: 'Counter withdrawal',
    calculated_balance: 15000
  },
  {

```

Figure 3.1. Successful mongodb queries

Challenges:

- Designing the same business logic in two different paradigms.
- Ensuring data consistency between the two separate systems.

Future Improvements:

- Implementing real-time synchronization between Microsoft Server SQL and MongoDB.
- Adding a front-end application that can connect to either database.
- Performance optimization and security enhancements.

References

- [1] R. Elmasri and S. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2017.
- [2] “ER Diagram of Bank Management System,” GeeksforGeeks, Jul. 2025. [Online]. Available: <https://www.geeksforgeeks.org/dbms/er-diagram-of-bank-management-system/> (accessed Apr. 28, 2026).
- [3] Addis Ababa Science and Technology University, “Database Group Project Description & Guidelines,” Software Engineering Department, 2026.
- [4] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York: McGraw-Hill, 2020.
- [5] C. J. Date, *An Introduction to Database Systems*, 8th ed. Addison-Wesley, 2003.
- [6] MongoDB Inc., “MongoDB Documentation: Data Modeling,” 2025. [Online]. Available: <https://www.mongodb.com/docs/manual/core/data-modeling/> (accessed May 10, 2026).
- [7] Oracle Corporation, “MySQL 8.0 Reference Manual,” 2025. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/> (accessed May 12, 2026).

Annex

Annex A: Survey Questions

Addis Ababa Science and Technology University

Bank Management System for Gebar Commercial Bank

Group 5

To better understand the existing challenges in the banking system, the following survey questions were prepared(for gathered answers refer, /annex/survey_questions.pdf):

Section 1: Customer Survey

1. How satisfied are you with the current account opening and registration process? (Very Satisfied / Satisfied / Neutral / Dissatisfied / Very Dissatisfied)
2. How long does it usually take to complete a deposit or withdrawal transaction at the branch? (Less than 5 minutes / 5–15 minutes / 15–30 minutes / More than 30 minutes)
3. Have you ever experienced errors in your account balance or transaction records? (Yes / No) If yes, please explain: _____
4. How easy is it to obtain an account statement or loan status update? (Very Easy / Easy / Difficult / Very Difficult)
5. What challenges do you face when applying for a loan? (Select all that apply) ☐ Long processing time ☐ Too much paperwork ☐ Lack of transparency ☐ Others:

6. Would you prefer a system that allows you to view your transaction history and account balance instantly? (Yes / No / Not Sure)
7. How important is it for the bank to provide digital access to your banking information? (Very Important / Important / Neutral / Not Important)

Section 2: Bank Staff Survey

8. How often do you face difficulties retrieving customer or account information quickly?
(Daily / Frequently / Occasionally / Rarely)
9. Does the current system cause duplication of customer or transaction data across departments? (Yes / No / Not Sure)
10. How would you rate the current loan approval and tracking process? (Very Efficient / Efficient / Inefficient / Very Inefficient)
11. What are the most common problems you encounter when processing transactions?
(Please specify): _____
12. Would a well-designed automated database system reduce your daily workload?
(Strongly Agree / Agree / Neutral / Disagree)

Section 3: Management Survey

13. How effective is the current system in generating timely and accurate management reports? (Very Effective / Effective / Not Effective)
14. What are the major data-related challenges affecting decision-making in the bank?
(Please specify): _____
15. Do you believe implementing a modern hybrid database system (relational + NoSQL) would improve overall bank operations? (Yes / No / Not Sure)
16. What key features would you like to see in the proposed Bank Management System?
(Open-ended): _____

Annex B: Sample Forms

Bank Management System for Gebar Commercial Bank

Addis Ababa Science and Technology University

Group 5

The following input forms were designed to facilitate accurate and efficient data entry into the system. Sample data has been provided for illustration purposes. For the entire input forms, refer to /annex/sample_forms.pdf.

1. Branch Registration Form

Field	Sample Data
Branch ID	B001
Branch Name	Main Branch
Address	Addis Ababa
Phone Number	09`11000001
manager_id	E001

2. Account Opening Form

Field	Sample Data
Customer ID	C001
Full Name	Abebe Kebede
Gender	Male
Phone Number	0911223344
Address	Addis Ababa
Registration Date	2026-01-05
Branch ID	B001
DOB	1965-11-20

Email	abebe.bekele@example.com
NationalID	7548658496973896

3. Employee Application Form

Field	Sample Data
Employee ID	E001
Full Name	Tsegaye Endale
Position	Manager
Email	tsegaye@gcb.m
Phone Number	0913000001
Salary	25000
Branch ID	B001
Hire Date	2020-03-15

4. Account Creation Form

Field	Sample Data
Account Number	A1001
Customer ID	C001
Branch ID	B001
Account Type	Savings
Initial Balance	5000
open_date	2026-01-11
status	Active

Annex C: Sample SQL Schema;

```
USE GEBAR_COMMERCIAL_BANK;
GO
DROP TABLE IF EXISTS Branch;
GO
CREATE TABLE Branch(
branch_id VARCHAR(10) NOT NULL,
branch_name VARCHAR(50) NOT NULL,
address VARCHAR(200),
phone_number VARCHAR(20),
manager_id VARCHAR(20),
CONSTRAINT pk_Branch PRIMARY KEY (branch_id)
);
GO
```